STATE-AWARE COMPUTING

Exact local reuse for repeated system intervention

RESEARCH NOTE / VERSION 3.0 / JULY 2026

State-Space Programming Methodology (SSPM) is a restricted state-transition runtime for repeated interventions on a shared system. It keeps a validated baseline state resident, edits a local target, propagates the complete declared dependency closure, and reuses unaffected values. If locality disappears, the runtime falls back to full replay.

SSPM shares resident state across repeated local interventions, propagates complete declared dependency closures, and falls back to full replay when locality disappears. Under declared semantics, incremental execution is from-scratch consistent; sparse closures often reduce CPU decision cost while dense, cold, and materialization-heavy regimes do not.

01 / TRANSITION SEMANTICS

Let x_t be sufficient declared state, u_t the exogenous input, and F the ordered transition: $\mathbf{x}_{t+1} = F(\mathbf{x}_t, \mathbf{u}_t)$. A from-scratch rollout produces a resident baseline trajectory. An intervention changes selected state, controls, or parameters. SSPM computes the transitive affected closure induced by declared reads, writes, ordering, and coupling. Values inside the closure are recomputed; unaffected values are reused.

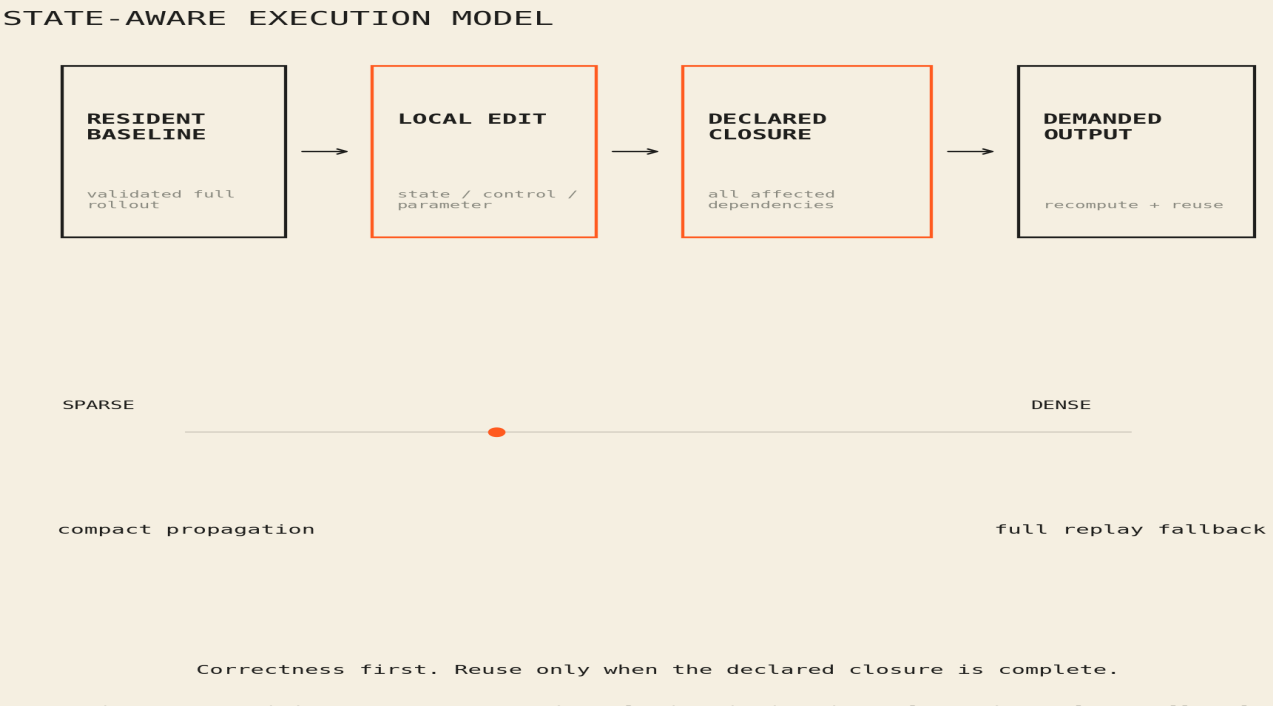

Figure 1. The compact path is correctness-preserving only when the dependency closure is complete. Full replay remains an explicit runtime path.

02 / CONDITIONAL EXACTNESS

Compact execution equals full replay when: (1) declared state is sufficient; (2) every read-after-write and cross-state dependency is represented; (3) the closure contains every value reachable from the edit over the requested horizon; (4) reused values share transition semantics, parameters, and inputs; and (5) output reconstruction is complete. This is a conditional semantic result, not a guarantee for hidden state or side effects.

The performance condition is equally explicit: $\mathbf{T}_{\text{closure}} + \mathbf{T}_{\text{compact}} + \mathbf{T}_{\text{materialize}} < \mathbf{T}_{\text{full}}$. Dense closures, cold compilation, transfer, or broad materialization can erase the benefit even when semantics remain exact.

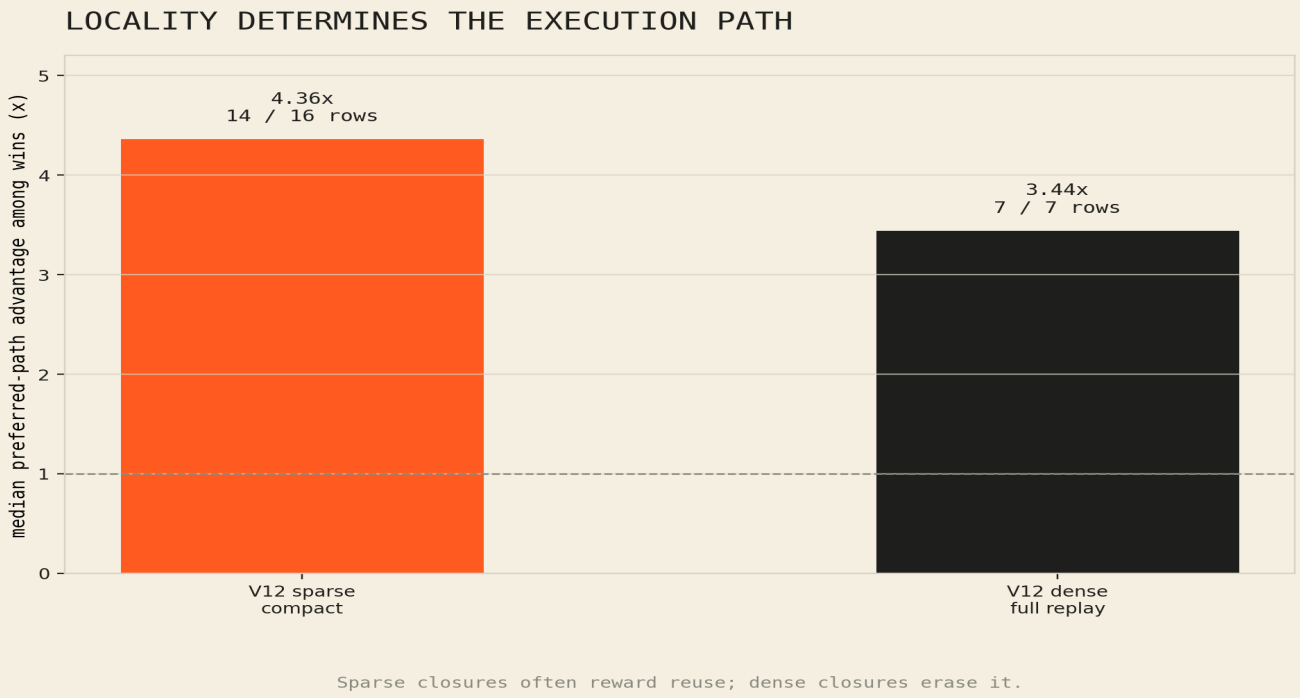

Figure 2. V12 is the clean paired result: compact won 14/16 sparse rows at 4.36x median among wins; full replay won 7/7 dense rows at 3.44x median.

03 / NOVELTY BOUNDARY

The work is adjacent to incremental and self-adjusting computation, dynamic dependence graphs, and differential data systems. SSPM's narrower focus is transition-oriented numerical systems with explicit state semantics, local intervention, demanded-output reconstruction, and a measured full-replay fallback. The source frontend covers four supported grammar families across 60 parameterized programs; it is not a general compiler or 60 independent applications.

Correcting affine classification so nonlinear products remain residual did not change the frozen V13 corpus: 40 parameterized programs are accepted, 20 reject explicitly, and accepted cases retain zero maximum error.

04 / EVIDENCE AND DISPOSITION

EVIDENCE	RESULT	BOUNDARY
V11 sparse	2.32x median / 5 qualifying rows	Broad median is 1.82x slower
V12 sparse	14/16 wins; 4.36x median	Decision path, among wins
V12 dense	Full wins 7/7; 3.44x median	Fallback evidence
V13 resident	23/24 wins; 5.58x median	Resident decision only
V13 workflow	21/24 wins; 4.43x median	Measured end-to-end boundary

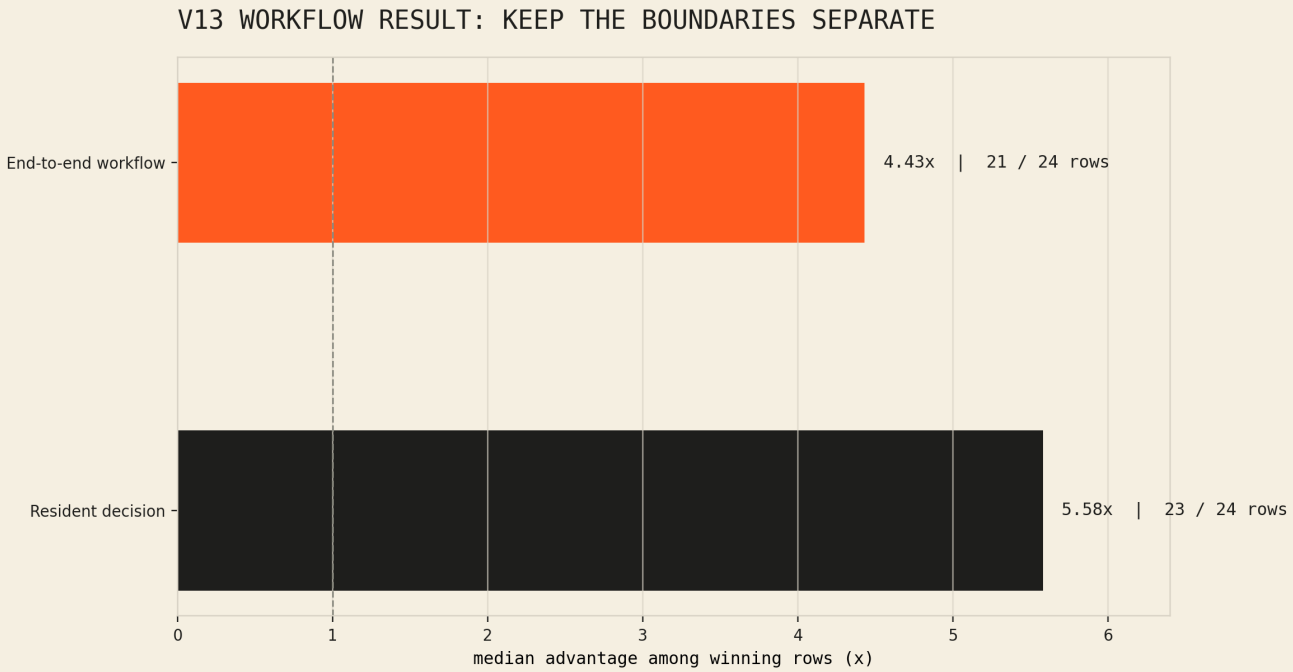

Figure 3. Resident-only and end-to-end V13 results are distinct claims and must remain separate.

NEGATIVE RESULTS

Cold JIT cost did not amortize through 10,000 tested decisions. V13 contains no dense rows. Differential Dataflow and isolated reproduction remain incomplete; CUDA/Triton and multi-GPU evaluation are deferred. The V13 classifier consumes realized closure after compact execution, so it is a post-hoc locality classifier rather than a deployable pre-execution selector.

DISPOSITION

The evidence supports state-aware computing as a bounded execution method: share a resident baseline, recompute the complete affected closure, reconstruct only what is demanded, and use full replay when locality or economics disappear. It does not support arbitrary-system linearization, hidden-state reuse, or universal incremental speed.

REFERENCES / Acar et al., ACM TOPLAS 2006 and 2009; McSherry et al., Differential Dataflow, CIDR 2013. Full citations, sanitized evidence, limitations, and executable source: github.com/bluetopazz/state-space-programming-research